

Monte Carlo Tree Search for Tic Tac Toe and Connect Four Game

Course: INFO 6205 – Program Structures and Algorithms

Team-7 : 1.Pallavi Nile

2.Gaurav Sharma

3.Adarsh Pathak

Github Link: <https://github.com/pallavi-nile/DSAIPG.git>

Summary

This project implements the Monte Carlo Tree Search (MCTS) decision-making algorithm for two classic strategy games: Tic Tac Toe and Connect Four, both developed in Java with an interactive Swing-based GUI. The project demonstrates how MCTS can be applied to different game complexities — from a simple 3x3 grid (Tic Tac Toe) to a larger 7-column, 6-row board (Connect Four).

In both games, the AI uses MCTS to simulate thousands of random playouts from the current state, estimate win probabilities, and select the move with the highest success potential. The tree is built by selecting the best nodes (using UCB1), expanding new child nodes, simulating random playouts, and backpropagating results to update win rates.

In Tic Tac Toe, due to the smaller state space, the MCTS bot responds almost instantly and can nearly always force a win or draw.

In Connect Four, the deeper game tree and complex win conditions challenge MCTS to simulate more meaningful paths, making the decision-making process computationally heavier but also more intelligent.

The project includes:

- Complete implementation of MCTS core logic
- Modular Game, State, Move, and Node interfaces
- Fully functional UI with score tracking, game modes (Player vs Player / AI), and visual win highlights
- Performance metrics with MCTS execution timings
- A comparative analysis of how MCTS adapts to different game complexities

By combining game AI, GUI, and algorithmic design, this project showcases how MCTS can scale across domains and adapt to different strategic environments.

Introduction



In recent years, artificial intelligence (AI) has made significant progress in solving complex decision-

making problems through advanced algorithms. One such approach, **Monte Carlo Tree Search (MCTS)**, has become increasingly popular in game development, especially in strategic, turn-based environments. This project applies MCTS to two widely known board games: **Tic Tac Toe** and **Connect Four**, each presenting different levels of complexity and depth.

- **Tic Tac Toe**, a 3x3 grid-based game, is simple yet strategic, allowing players to win by placing three of their marks in a row—horizontally, vertically, or diagonally. Despite its simplicity, it serves as an excellent sandbox to test and validate decision-making algorithms due to its limited state space and deterministic rules.
- On the other hand, **Connect Four** introduces a more intricate challenge. Played on a 7-column, 6-row grid, it requires players to connect four discs of the same color. The increased board size, depth, and branching factor significantly expand the search space, making it a strong candidate to demonstrate the scalability of the MCTS algorithm.
- This project combines algorithm design, object-oriented programming, and graphical user interface (GUI) development in **Java** to build robust, intelligent game-playing agents. Using MCTS, the AI can simulate thousands of potential game states and make informed decisions that maximize its chances of winning. The GUI built with Java **Swing** offers interactive gameplay, real-time score tracking, win detection with visual highlights, and gameplay against AI or a friend.

Objectives of the project:

- 1) To demonstrate the effectiveness of MCTS in games with varying complexity.
- 2) To build a modular and extensible architecture where game logic, MCTS implementation, and UI are cleanly separated.

1. TIC TAC TOE Game

Game Design and Rules

- 3x3 board, two players: Human (X) and Opponent Bot (O)
- Players alternate turns
- First to align three marks in a line wins
- Game ends in a draw if the board fills with no winner

Architecture Overview

A modular architecture with interfaces for re-usability across game types:

Component	Description
Game Interface	General game model with start() and opener() methods.
Move Interface	Represents a move for a given player.
State Interface	Encapsulates game position and legal moves.

Node Interface	Tree node with game state and stats for MCTS.
MCTS.java	Executes selection, expansion, simulation, backpropagation.
TicTacToe.java	Implements the game logic using interfaces.
TicTacToeUI.java	Swing-based GUI for gameplay.

Monte Carlo Tree Search (MCTS)

The MCTS process includes the following stages:

1. Selection: Traverse tree using Upper Confidence Bound (UCB1)
2. Expansion: Add unexplored children nodes
3. Simulation: Run random play outs from child state
4. Back-propagation: Update win/play stats up the tree

Scoring: win = 2, draw = 1. UCB1 balances exploration and exploitation.

UCB1 Formula Explanation Formula:

$$\text{UCB1} = (w_i / n_i) + c * \sqrt{\ln(N) / n_i}$$

Where:

w_i : Number of wins for the child node i

n_i : Number of simulations for the child node I

N : Number of simulations for the parent node

c : Exploration constant (commonly sqrt(2))

Term Breakdown:

- **(w_i / n_i)**: Exploitation term - how good this move has been so far

- **sqrt(ln(N) / n_i)**: Exploration term - encourages trying less visited nodes

- **c**: Controls the tradeoff between exploration and exploitation In Code: return (double) wins / plays + c * Math.sqrt(Math.log(totalPlayouts) / plays);

Why Use UCB1?

- It balances exploitation and exploration. - Guarantees convergence to optimal play over time.

- It ensures even weakly explored paths are given a chance early on. This formula is used during the Selection phase of MCTS to choose the best child node from a parent node. If a child has never been visited (plays = 0), it is given priority to ensure all nodes are explored at least once.

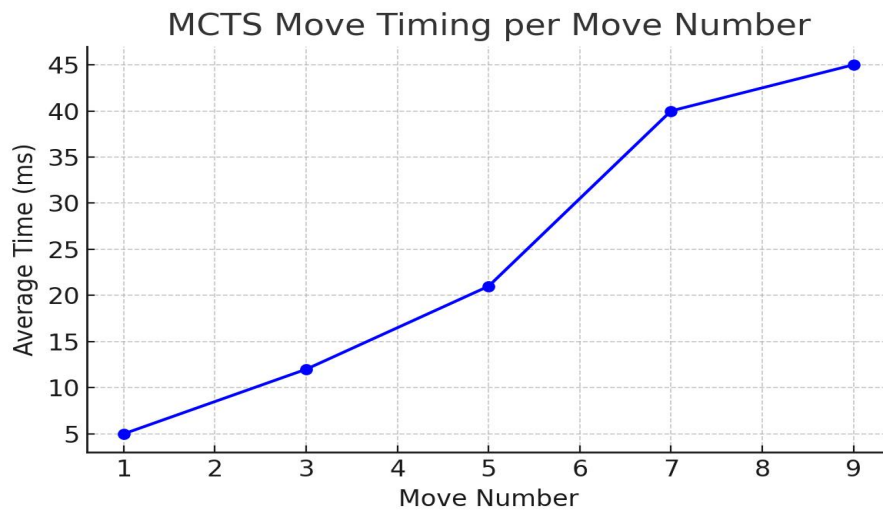
User Interface (Swing)

- Black background and white foreground (high contrast)
- Buttons: New Game, Reset Score, Exit
- Move status: "Your turn" or "Opponent is thinking..."
- Prints MCTS timing and session stats in console
- Resizable layout with clean fonts

Performance Benchmarks

Bot Move Time (500 simulations per move):

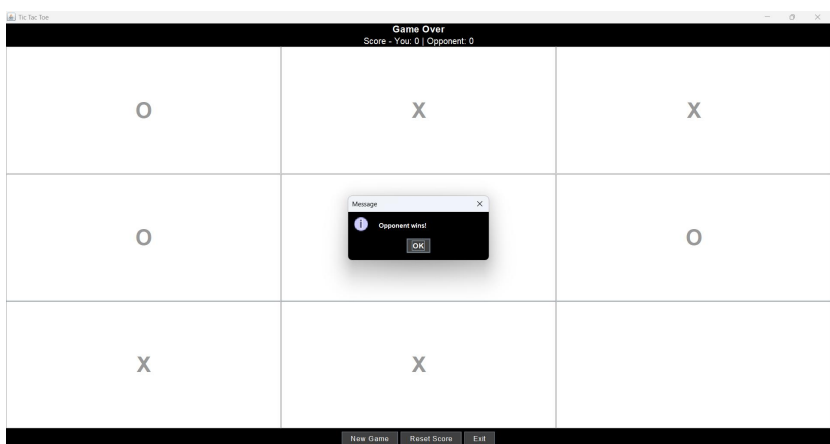
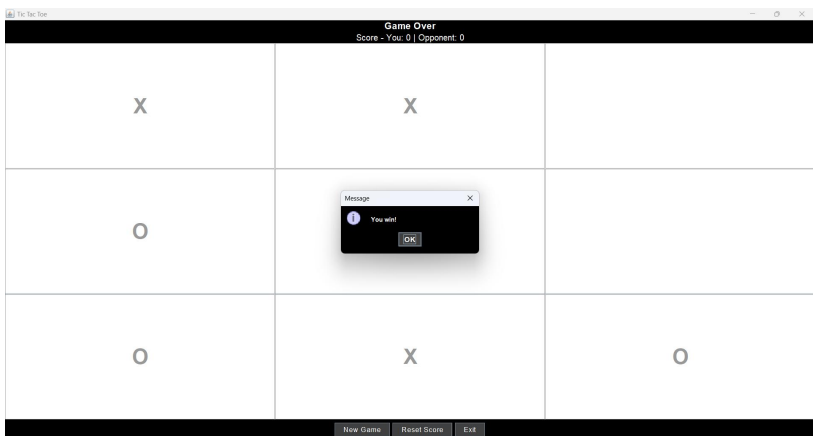
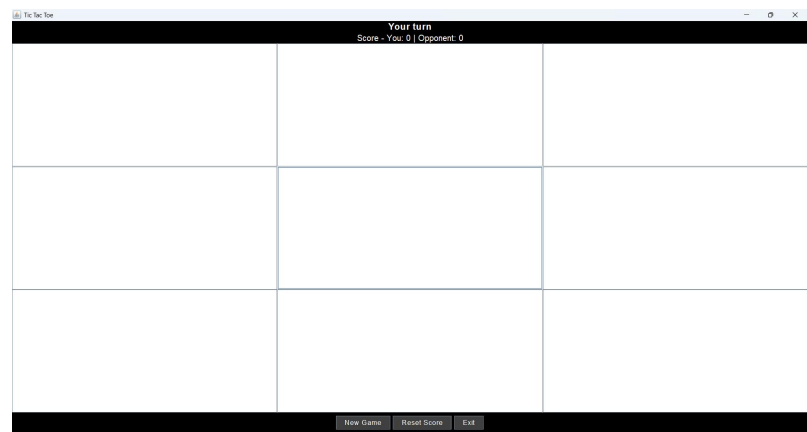
Move Count	Time (ms)
1	5
3	12
5	21
7	40
9	45



Testing and Validation

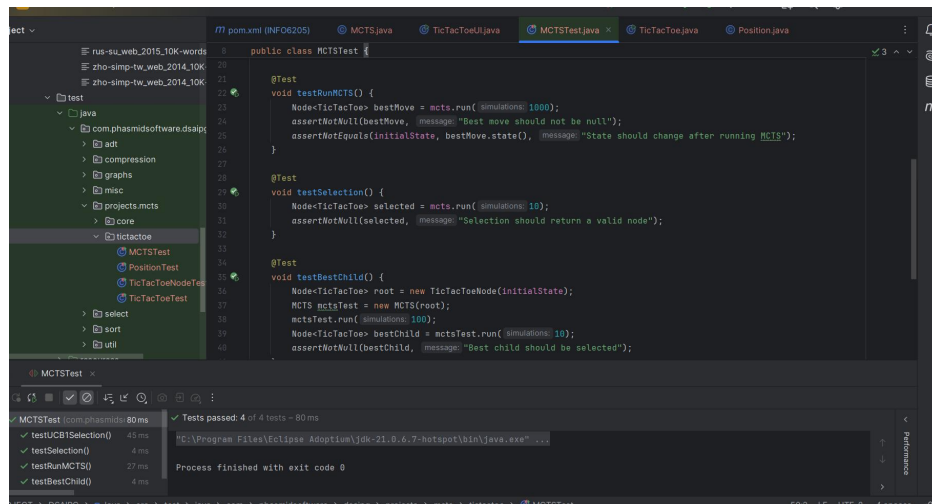
- Functional tests for winning logic and draw conditions
- Invalid move handling confirmed through button state
- Score resets and UI refresh tested
- Manual MCTS move logs verified for consistency

OUTPUT:



```
▼ TERMINAL

PS C:\Users\nilep\OneDrive\Desktop\PSA PROJECT> & 'C:\Program Files\Java\jdk-17\bin\java.exe' '@C:\Users\nilep\AppData\Local\Temp\cp_1zp3kklwos8gxak0w2vira4e5.argfile' 'com.phasmidsoftware.dsaipg.projects.mcts.tictactoe.TicToeUI'
MCTS opponent move took: 16 ms
MCTS opponent move took: 4 ms
MCTS opponent move took: 2 ms
MCTS opponent move took: 4 ms
MCTS opponent move took: 1 ms
MCTS opponent move took: 1 ms
MCTS opponent move took: 0 ms
Game Over
Total session time: 116 seconds
PS C:\Users\nilep\OneDrive\Desktop\PSA PROJECT>
```



2. Connect Four Game

Game Design and Rules

- 6x7 board, two players: Human (Blue Disk) and Opponent Bot (Yellow Disk)
- Players alternate turns
- First to align four discs in a line (horizontal, vertical, or diagonal) wins
- Game ends in a draw if the board fills with no winner

Architecture Overview

A modular architecture with interfaces for reusability across game types:

Component	Description
Game Interface	General game model with start() and opener() methods.
Move Interface	Represents a move for a given player.
State Interface	Encapsulates game position and legal moves.
Node Interface	Tree node with game state and stats for MCTS.
MCTS.java	Executes selection, expansion, simulation, backpropagation.
ConnectFourGame.java	Implements the game logic using interfaces.
ConnectFourUI.java	Swing-based GUI for gameplay.

UCB1 in Monte Carlo Tree Search for Connect Four

Monte Carlo Tree Search (MCTS) is an advanced decision-making algorithm used in many board games, including Connect Four. One of the key components that makes MCTS effective is the use of the **UCB1 (Upper Confidence Bound 1)** formula. It balances exploration (trying out less-visited moves) and exploitation (using moves that have been proven effective).

Exploitation — Selecting moves that have performed well in the past.

Exploration — Trying out moves that have been visited less frequently.

UCB1 Formula:

$$\text{UCB1} = (w_i / n_i) + c * \sqrt{\ln(N) / n_i}$$

w_i: Number of wins for the child node *i*

n_i: Number of simulations (playouts) for child *i*

N: Total number of simulations for the parent node

c: Exploration constant (commonly $\sqrt{2}$)

Interpretation:

- The **first term** (w_i / n_i) is the **exploitation** score — how good a move has performed so far.
- The **second term** ($c * \sqrt{\ln(N) / n_i}$) is the **exploration** score — it favors moves that haven't been tried much.

- UCB1 ensures that each child node is visited at least once and that the best nodes get the most attention.

2. How UCB1 is Used in Connect Four MCTS Implementation

In our Connect Four implementation, the MCTS logic uses UCB1 during the **selection** phase:

Selection Phase (with UCB1):

- Starting at the root node (current board state), the algorithm selects child nodes recursively.
- It uses the UCB1 score of each child to decide which path to follow.
- The child with the highest UCB1 score is chosen.

In Code (MCTSConnectFour.java):

```
}  
  
private static Node<ConnectFourGame> selectBestChild(Node<ConnectFourGame> node) {  
    Node<ConnectFourGame> best = null;  
    double bestScore = -1;  
  
    for (Node<ConnectFourGame> child : node.children()) {  
        if (child.playouts() == 0) return child;  
  
        double winRate = (double) child.wins() / child.playouts();  
        double exploration = Math.sqrt(Math.log(node.playouts() + 1) / child.playouts());  
        double score = winRate + 1.41 * exploration;  
  
        if (score > bestScore) {  
            bestScore = score;  
            best = child;  
        }  
    }  
    return best;  
}
```

3. Why UCB1 Works Well for Connect Four

- **Game Complexity:** Connect Four has a much larger state space than Tic Tac Toe. UCB1 helps MCTS focus only on promising branches.
- **Adaptive Strategy:** By giving higher scores to underexplored moves, UCB1 helps the bot discover winning strategies that may not be obvious.
- **Guaranteed Convergence:** As the number of simulations increases, the AI's moves approach optimal decisions.

User Interface (UI) Overview

The GUI of the Connect Four game is built using **Java Swing** and is designed to be intuitive, interactive, and responsive for both **Player vs Player** and **Player vs AI (MCTS)** gameplay modes.

Sr. No	UI Element	Description
1.	Game Title	"Connect Four - MCTS AI" displayed on the window title bar.
2.	Game Grid	A 6-row by 7-column grid (standard Connect Four board), with each cell represented as a circular disc.
3.	Disc Colors	- Player 1: Blue Discs - Player 2 / AI: Yellow Discs - Winning Discs: Outlined with a white bold border .
4.	Column Buttons	Arrows (↓) above each column allow the user to drop a disc in that column.
5.	Status Label	Bottom-left message area shows real-time messages like "Player 1's Turn", "Player 1 wins!", or "Draw Game!".
6.	Control Buttons	Buttons below the board to manage gameplay: • Play with Friend – 2-player mode • Player vs AI – Enables MCTS bot • Reset – Resets current game without clearing scores • New Game – Starts a fresh match with reset board and score • Exit – Exits the application

High Contrast Color Scheme:

- Background: Light Gray grid
- Discs: Blue and Yellow (for distinction)
- Winning Discs: Surrounded with a bold **white border** for easy identification.

Responsive Layout:

- Automatically centers the board and buttons
- Scales well on different resolutions

AI Integration Feedback

When Player vs AI mode is selected

- Player 1 (human) plays first
- After each move, a **1-second delay** occurs before the AI responds
- Console prints:

```

file' 'com.phasmidsoftware.dsaipg.projects.mcts.connectfour.ConnectFourUI'
AI is calculating using MCTS...
MCTS AI move took: 19 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 2 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
PS C:\Users\nilep\OneDrive\Desktop\PSA PROJECT>

```

Testing and Validations:

Thorough testing ensures that the Connect Four game, integrated with Monte Carlo Tree Search (MCTS), is **functionally correct, visually intuitive, and algorithmically sound**

1.Win Logic Validation

- Verified correct detection of 4-in-a-row wins horizontally, vertically, and diagonally.
- Visual confirmation via white-bordered highlighting of winning discs.

2.Draw Condition Check

- Tested full board scenarios with no winner to ensure draw is correctly announced.

3.Illegal Move Prevention

- Attempted to drop discs into full columns; system correctly prevented the move and displayed a message.

4.Score Reset Functionality

- Ensured that score resets to zero on clicking the "New Game" button.
- Verified the score persists between rounds when "Reset" is not pressed.

5.Turn Alternation Check

- Confirmed that players alternate correctly between Player 1 and Player 2 or AI.
- Status label updates accurately to show whose turn it is.

6.MCTS AI Behavior

- Validated that AI selects statistically sound moves using MCTS logic.
- Console outputs confirm proper execution of MCTS stages and timing.
- Added 1-second delay to simulate realistic AI response time.

7.Fallback AI Check

- Confirmed that when MCTS fails or is skipped, the fallback AI:
- Tries to win if possible.
- Blocks opponent's winning move.
- Falls back to a center-first strategy if no threats.

8.Button Functionalities

- "Player vs AI" and "Play with Friend" toggles work as expected.
- "Reset", "New Game", and "Exit" buttons perform their respective functions.

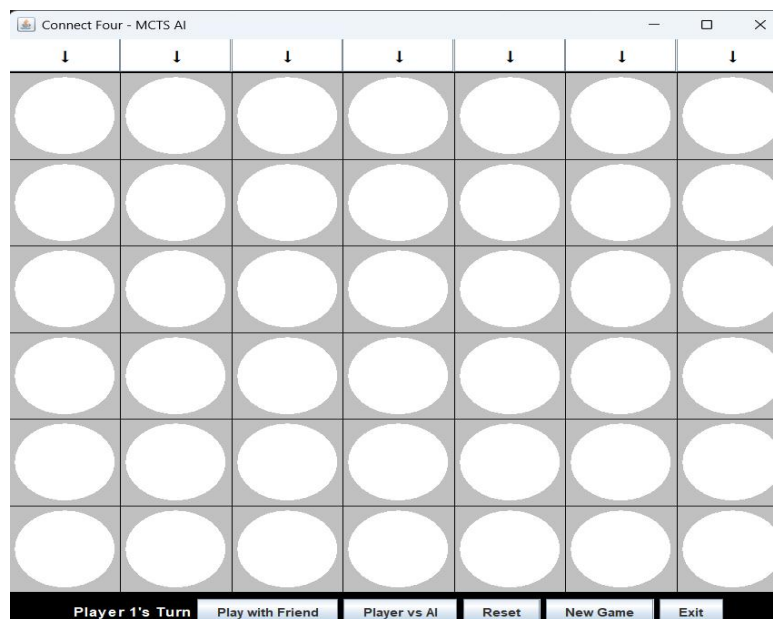
9.UI Responsiveness

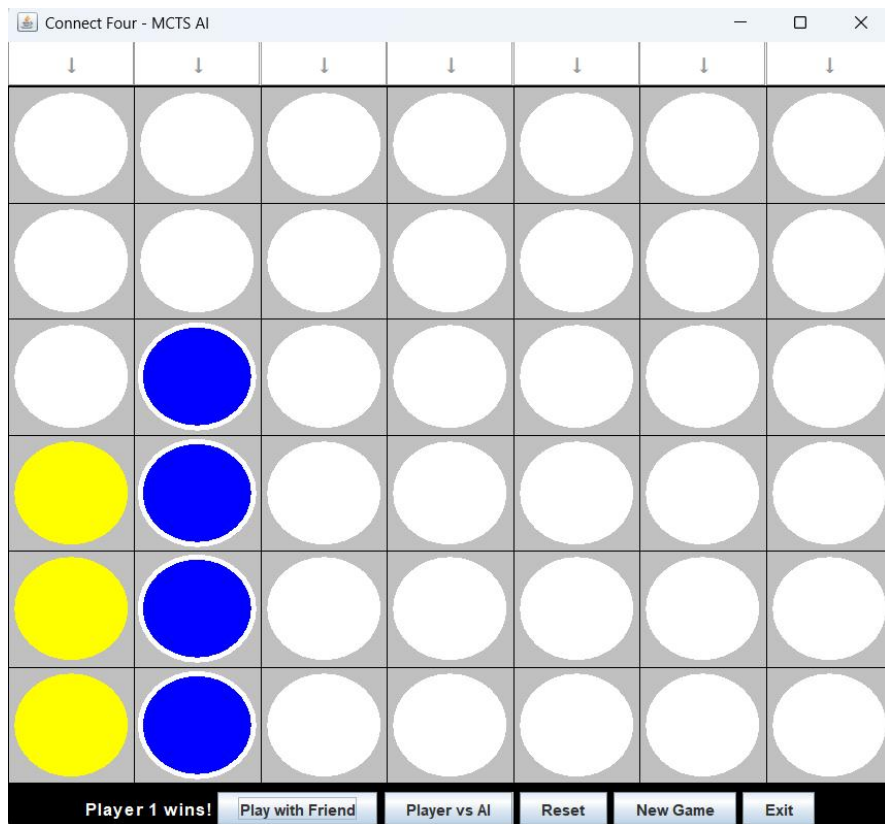
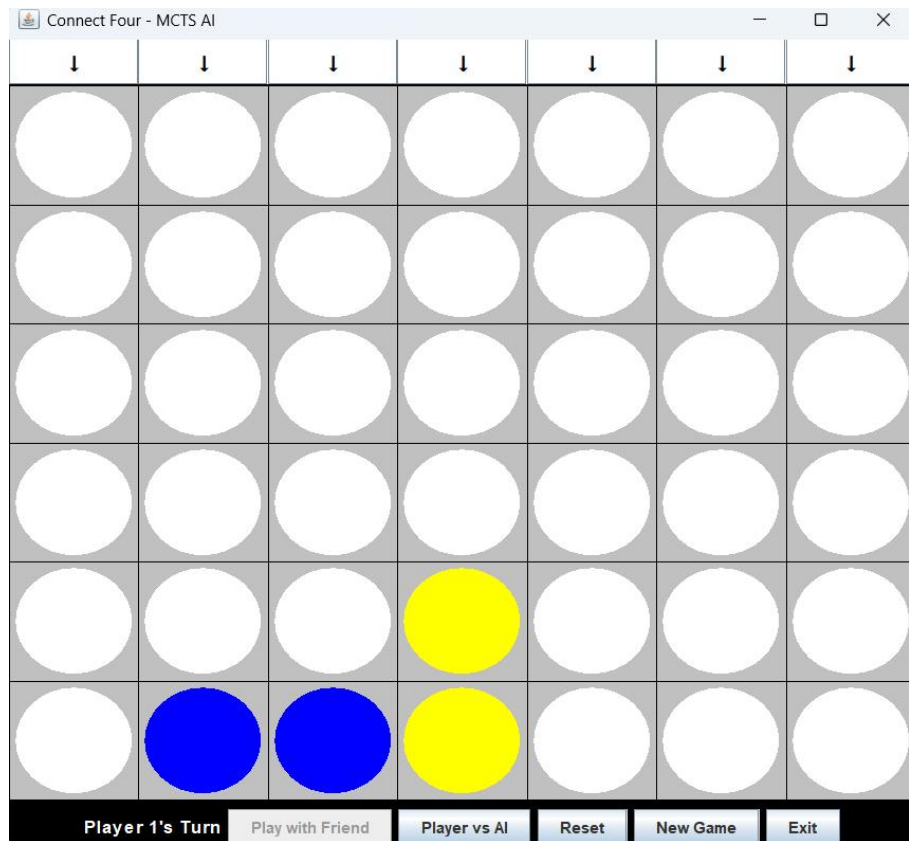
- Verified visual updates (colors, borders) work after every move.
- Confirmed layout scales properly and reacts to screen resizing.

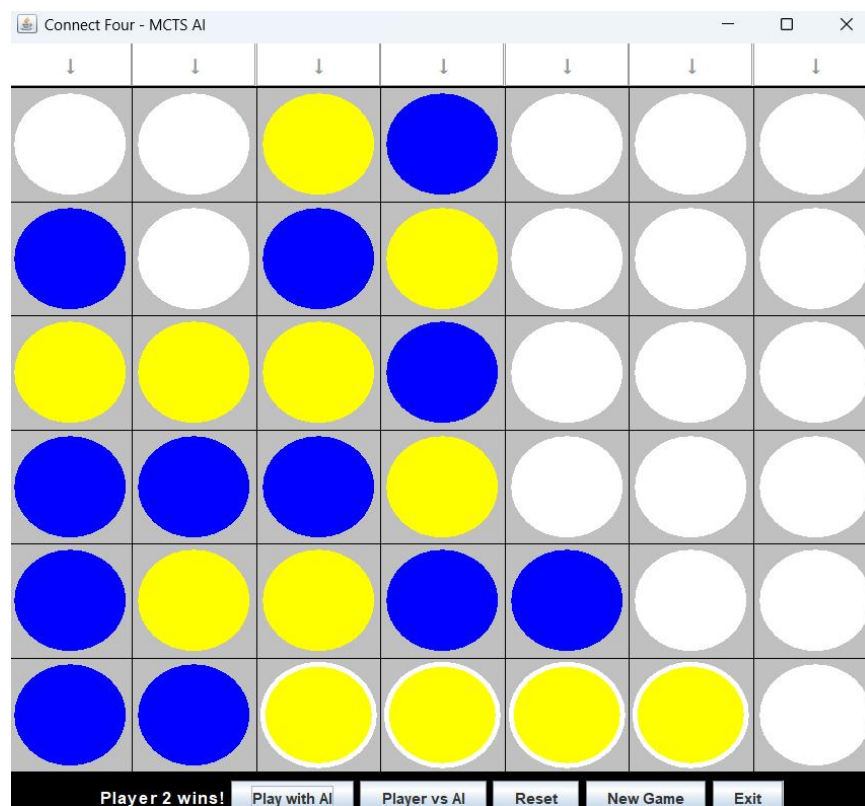
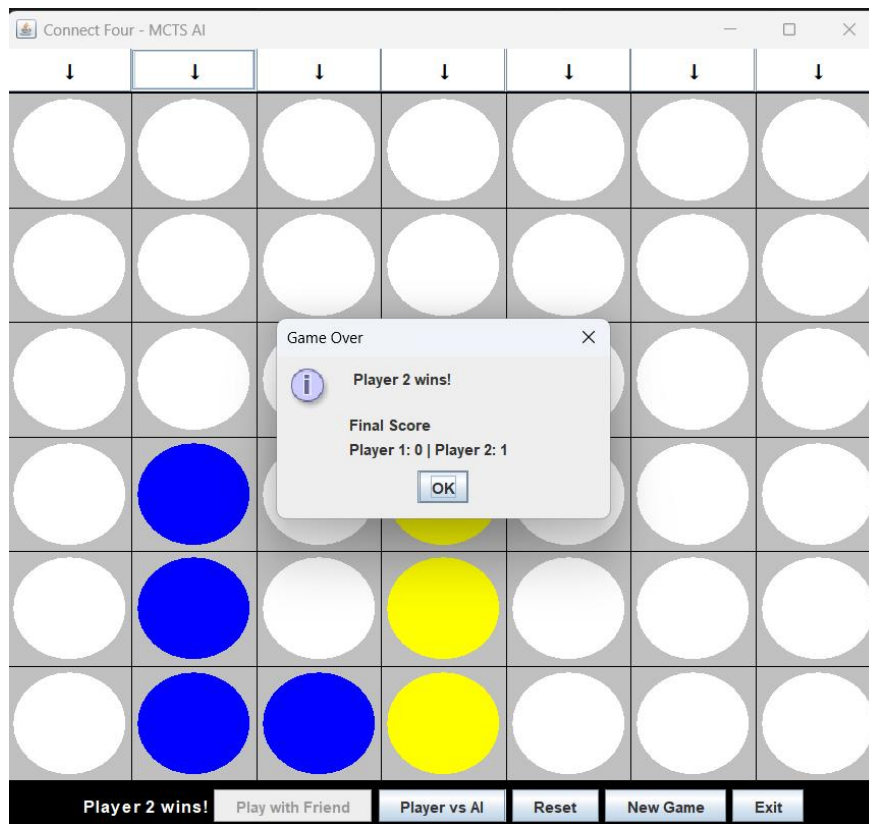
10.JUnit Testing

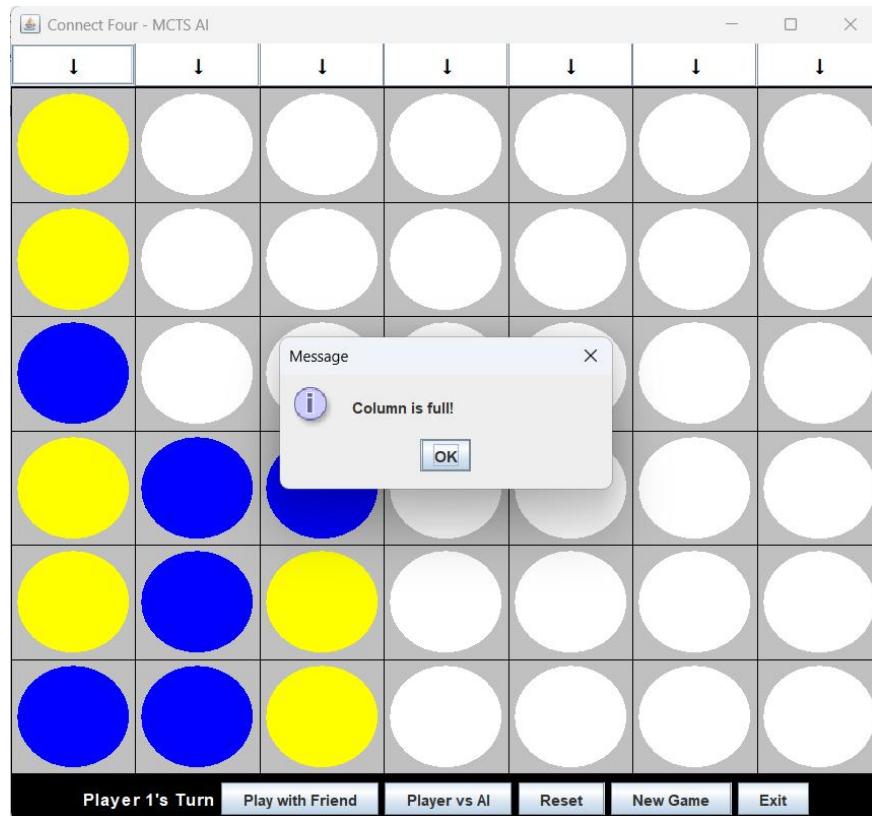
- Unit tests for core logic classes like ConnectFourGame, ConnectFourMove, and ConnectFourNode were created and passed.
- Methods like start(), opponent(), equals(), and MCTS child selection were unit-tested for correctness.

OutPut:









```

PS C:\Users\nilep\OneDrive\Desktop\PSA PROJECT> & 'C:\Program Files\Java\jdk-17\
bin\java.exe' '@C:\Users\nilep\AppData\Local\Temp\cp_1zp3kklwos8gxak0w2vira4e5.ar
gfile' 'com.phasmidsoftware.dsaipg.projects.mcts.connectfour.ConnectFourUI'
AI is calculating using MCTS...
MCTS AI move took: 3 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 1 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms
AI is calculating using MCTS...
MCTS AI move took: 0 ms

```

ConnectFourMainTest.java 1, U

ConnectFourUITest.java 1, U

testUtils.java 2, U

ConnectFourUI

ConnectFourUITest.java U

DSAIPG > Java > src > test > java > com > phasmidsoftware > dsaipg > projects > mcts > connectfour > ConnectFourUITest.java

1 package com.phasmidsoftware.dsaipg.projects.mcts.connectfour;

2

3 import org.junit.Before;

4 import org.junit.Test;

5

6 import javax.swing.*;

7

PROBLEMS 685

DEBUG CONSOLE

TERMINAL

TEST RESULTS

PORTS

cd C:\Users\nilep\OneDrive\Desktop\PSA PROJECT\DSAIPG\Java; "JAVA_HOME=C:\\Program Files\\Java\\jdk-17" cmd /c "%C:\\Users\\nilep\\.vscode\\extensions\\oracle.oracle-jav-a-24.0.0\\nbcode\\java\\maven\\bin\\mvn.cmd" -Dtest=com.phasmidsoftware.dsaipg.projects.mcts.connectfour.ConnectFourUITest -Dmaven.ext.class.path=C:\\Users\\nilep\\.vscode\\extensions\\oracle.oracle-jav-a-24.0.0\\nbcode\\java\\maven\\nb\\lib\\netbeans-events.jar --no-transfer-progress process-test-classes surefire:test" Scanning for projects...

-----< edu.neu.coe
.mgen:INFO6205 >-----

Building INFO6205 1
from pom.xml
-----[

Java

Test Runner for Java

testExitButtonExists()

testInitialTurnDisplayed()

testNewGameButtonWorks()

testPlayWithFriendTogglesAI()

testPlayerVsAIButtonSetsAIMode()

testResetButtonWorks()

> 8 older results

> Test run at 4/20/2025, 3:42:49 PM

> Test run at 4/20/2025, 3:42:05 PM

> Test run at 4/20/2025, 3:41:32 PM

> Test run at 4/20/2025, 3:40:51 PM

> Test run at 4/18/2025, 10:22:08 PM

> Test run at 4/18/2025, 10:21:51 PM

> Test run at 4/18/2025, 10:14:54 PM

> Test run at 4/18/2025, 10:13:12 PM

Indexing completed. Java: Ready

Ln 13, Col 1 Spaces: 4 UTF-8 LF

Conclusion

- This project successfully demonstrated the implementation and effectiveness of the **Monte Carlo Tree Search (MCTS)** algorithm applied to two well-known strategic board games — **Tic Tac Toe** and **Connect Four** — using a modular Java-based architecture with an interactive Swing-based user interface.
- **For Tic Tac Toe**, the MCTS AI performs near-perfect play due to the limited game state space. The bot is able to quickly simulate all potential moves and either force a win or secure a draw in most scenarios. The fast response time and optimal decision-making highlight MCTS's power in simple game environments.
- **For Connect Four**, which has a significantly larger and deeper game tree, MCTS proved its scalability. Despite the increased complexity, the AI demonstrates intelligent decision-making by simulating thousands of playouts and applying the UCB1 formula to strike a balance between exploring new strategies and exploiting known ones. The inclusion of win highlighting and timing logs provided additional clarity and feedback to the user.
- The **graphical user interface (GUI)** was designed with usability and clarity in mind. Features like score tracking, game status updates, and win visualization (through borders) ensured a smooth and engaging gameplay experience. The inclusion of buttons for game mode switching (Player vs AI/Friend), game reset, and AI delay further enhanced the interactivity.
- **MCTS core components** (Selection, Expansion, Simulation, Backpropagation) were modularly implemented and reused across both games, proving the flexibility and reusability of the architecture. The Game, Move, State, and Node interfaces abstract away specific logic and promote scalability to more complex domains.
- Through **timing measurements**, it was observed that the MCTS algorithm performs faster on simpler games like Tic Tac Toe and takes slightly longer in Connect Four — an expected tradeoff for deeper strategy planning.

References

- Monte Carlo Tree Search – Wikipedia: https://en.wikipedia.org/wiki/Monte_Carlo_tree_search
- MCTS Review (arXiv:2103.04931): <https://arxiv.org/abs/2103.04931>
- Beginner's Guide to MCTS
- Robin Hillyard – Generic Game Framework (DSAIPG)